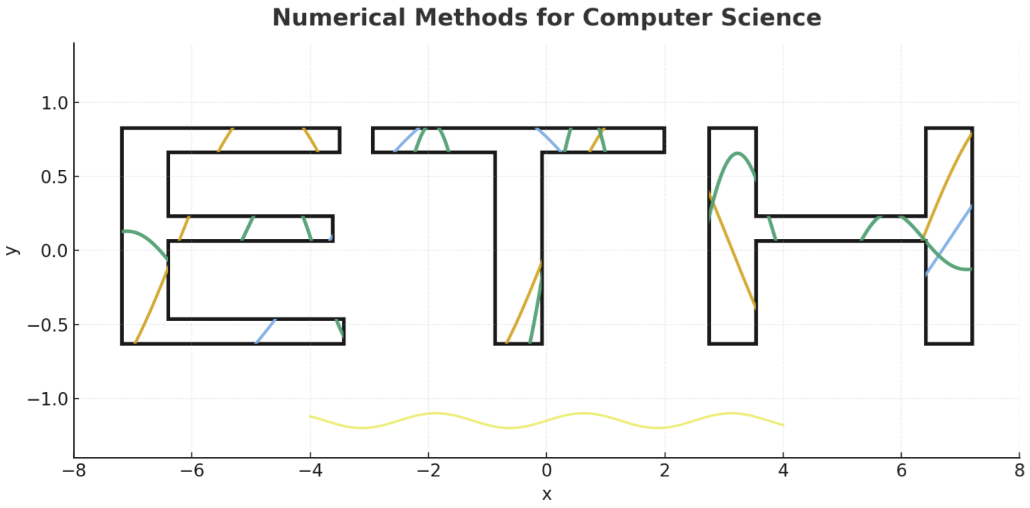

Weekly Summary — Week #2

ETH Zurich

DAVID Mihnea-Ștefan

mihdavid@ethz.ch

This document summarizes core topics discussed in Week 2. The following page lists the contents; the body includes concise statements, examples, and code snippets.

Contents

Contents

1 Introduction: Why Numerical Methods for Computer Science?

Computers are powerful, but they do not work with exact mathematics. Every number stored in a machine is represented with finite precision, and every algorithm we run is constrained by time and memory. As a result, most real-world problems in science, engineering, and data analysis cannot be solved exactly — they must be approximated.

Numerical methods provide the principles and tools to do this safely. They help us answer questions such as:

- How can we approximate a mathematical problem so that the error is small and quantifiable?
- How do we design algorithms that are not only correct in theory, but also stable in practice?
- How can we measure the cost of computation and choose efficient methods when data becomes large?

For computer scientists, these skills are essential. Whether you are training a machine learning model, simulating a physical system, or solving a linear system inside an optimization routine, you are relying on numerical algorithms. Understanding their foundations makes you a better programmer, a more careful data scientist, and a more reliable engineer.

In short: Numerical Methods are the bridge between mathematics and computation. They ensure that what we calculate with a computer actually reflects reality — within predictable error bounds, and at an acceptable computational cost.

2 Slow Start in Python

2.1 Recommendation

For this course, you should read the handout: *"Short Introduction to Python, NumPy, Matplotlib, etc."*
https://polybox.ethz.ch/index.php/s/dLjQej2BHbCJBwz

> Note
>
> This short document provides the basic background needed to follow the examples and write your own scripts. It covers essentials of Python syntax, numerical libraries, and simple plotting.

2.2 Working with Arrays

In Python, arrays can be native `list` objects. For numerical computing we typically use `NumPy arrays`, which are efficient and support vectorized operations.

Listing 1: From Python list to NumPy array and back

```
import numpy as np
L  = [1, 2, 3, 4]    # Python list
A  = np.array(L)     # NumPy array
L2 = A.tolist()      # back to list
```

> Note
>
> A NumPy array is homogeneous (single dtype) and contiguous in memory. This enables vectorized operations and speed. Prefer arrays over Python lists for numerical work.

2.3 Matrix Operations

Listing 2: Basic matrix operations in NumPy

```
import numpy as np
M = np.array([[1,2,3],
              [4,5,6],
              [7,8,9]])

M.T                 # transpose
np.diag(M)          # diagonal as 1D array
M.sum()             # sum of all elements
M.mean()            # mean of all elements
M.min(), M.max()    # min and max
np.trace(M)         # trace (sum of diagonal)
```

Note

Use `np.diag(M)` for the diagonal as a vector; use `np.diag(v)` to create a diagonal matrix from a vector `v`. The `trace` is invariant under cyclic permutations: $\mathrm{tr}(AB) = \mathrm{tr}(BA)$ when products are defined.

2.4 Vector Operations

Listing 3: Dot product

```
import numpy as np
x = np.array([1,2,3])
y = np.array([4,5,6])
x.dot(y)            # 32
```

Note

`u.dot(v)` (or `u @ v`) computes the inner product when both are 1-D. For 2-D arrays, `@` performs matrix multiplication. Elementwise multiply remains `u * v`.

2.5 Reshape and Blocks

Listing 4: Reshape and block matrices

```
import numpy as np
A = np.arange(1,13).reshape(3,4)
B = A.reshape(2,6)    # reshape

top    = np.hstack([np.eye(2),  np.ones((2,2))])
bottom = np.hstack([np.zeros((2,2)), np.eye(2)])
Block  = np.vstack([top, bottom])
```

Note

`reshape` uses row-major (C-order) by default. For explicit control, pass `order='F'` for column-major semantics. Prefer `np.block([...])` for readable block assembly over nested `hstack`/`vstack`.

2.6 Creating Arrays Reliably (zeros/ones/eye/diag, arange vs. linspace)

Prefer explicit constructors that avoid surprises (especially with floating steps).

Listing 5: Robust array constructors

```
import numpy as np
np.zeros((3,3)); np.ones((2,4)); np.eye(3)            # basic grids
np.diag([1,2,3])                                      # diagonal from
    vector
```

```
np.arange(0, 1, 0.1)          # beware of float steps (binary rounding)
np.linspace(0, 1, 11)         # robust endpoints: 0.0 ... 1.0 exactly
```

Note

Prefer `np.linspace` for floating ranges; it includes exact endpoints and avoids cumulative spacing drift. Use `np.arange` mainly for integer steps.

2.7 Indexing, Slicing: Views vs. Copies

Slices are usually *views* (no data copy). Assignments on views affect the original array.

Listing 6: Views vs copies

```
A = np.arange(9).reshape(3,3)
B = A[:, :2]          # view: shares memory
B[0,0] = 999
A[0,0]                # -> 999 (changed!)
C = A[:, :2].copy()   # explicit copy: independent
```

Note

Check sharing explicitly with `np.shares_memory(B, A)`. Views are fast and memory-friendly; use `.copy()` only when you need isolation.

2.8 Axes, Reductions, and `keepdims`

Control along which axis reductions act; preserve dimensions when needed for broadcasting.

Listing 7: Reductions by axis

```
X = np.arange(12).reshape(3,4)
X.sum(axis=0)                # column-wise (shape: (4,))
X.sum(axis=1)                # row-wise    (shape: (3,))
X.mean(axis=1, keepdims=True)# shape: (3,1) -- handy for broadcasting
```

Note

The axis you reduce over disappears by default. Use `keepdims=True` to keep reduced axes as size-1, which simplifies later broadcasting.

2.9 Broadcasting (Vectorization Essential)

Broadcasting aligns shapes without explicit loops.

Listing 8: Outer sum via broadcasting

```
x = np.linspace(0, 1, 5)      # shape (5,)
```

```
y = np.linspace(0, 1, 4)       # shape (4,)
M = x[:, None] + y[None, :]    # shape (5,4)
```

> Note
>
> `None` and `np.newaxis` are equivalent for inserting size-1 axes. Broadcasting rules compare shapes from the right; size-1 dimensions expand to match.

2.10 Elementwise vs. Matrix Products (`*` vs. `@`)

Use `*` for elementwise multiplication and `@` (or `dot`) for matrix products.

Listing 9: Elementwise vs matrix product

```
A = np.arange(4).reshape(2,2)
B = np.ones((2,2))
A * B       # elementwise
A @ B       # matrix product
A.dot(B)    # same as @
```

> Note
>
> `@` is associative but not commutative. For higher-rank arrays, prefer `np.matmul` (same as `@`) and broadcasting-aware operations.

2.11 Numerical Comparisons: `np.isclose/np.allclose`

Never rely on `==` with floating-point numbers; compare within tolerances.

Listing 10: Robust float comparisons

```
import numpy as np
a = 0.1 + 0.2
b = 0.3
a == b                          # False (expected)
np.isclose(a, b), np.allclose([a], [b])
```

> Note
>
> Control tolerances with `rtol` (relative) and `atol` (absolute). A good default is often `rtol=1e-8, atol=1e-12` for float64-scale values.

2.12 Timing and Reproducibility (RNG)

Simple wall-clock timing and modern RNG usage.

Listing 11: Timing + reproducible RNG

```
import time, numpy as np
```

```
t0 = time.perf_counter()
np.sqrt(np.arange(100_000))
dt = time.perf_counter() - t0
print(f"{dt:.4f} sec")
```

> Note
>
> For micro-benchmarks in notebooks, prefer `%timeit`. Use `default_rng = np.random.default_rng` (NumPy 1.17+) with a fixed `seed` for reproducible results.

2.13 Reshape Details, `ravel/flatten`, and `np.block`

`ravel` returns a view when possible; `flatten` always copies.

Listing 12: Shape manipulation and block assembly

```
A = np.arange(12).reshape(3,4)
a_view = A.ravel()       # view if possible
a_copy = A.flatten()     # guaranteed copy

B = np.block([
  [np.eye(2),              np.ones((2,2))],
  [np.zeros((2,2)),        np.eye(2)]
])
```

2.14 Mini `linalg` Toolkit

Avoid forming explicit inverses; use solvers and norms.

Listing 13: Solve and norms

```
from numpy.linalg import norm, solve
A = np.array([[3., 2.],
              [1., 2.]])
b = np.array([5., 5.])
x = solve(A, b)         # preferred over inv(A) @ b
rel_res = norm(A@x - b) / norm(b)
```

> Note
>
> `solve(A,b)` is numerically safer and faster than `inv(A) @ b`. For least squares, use `np.linalg.lstsq` or `pinv` when appropriate (more details later).

3 Introduction to Numerical Analysis

Numerical Analysis studies how to approximate mathematical problems by algorithms that can actually be executed on a computer. In practice, exact solutions are rare; we aim for *good approximations* that are also *efficient to compute*.

Two aspects are central:

- **Accuracy:** the approximation should be close to the true solution, and the error should be quantifiable.
- **Efficiency:** the method should require a reasonable amount of time and memory.

In short, numerical analysis connects *ideas* (mathematics) with *implementation* (algorithms). The objective is to design methods that converge to the right solution while keeping computational cost under control.

3.1 Fundamental Aspects

The lecture emphasized:

- **Convergence:** sequences and algorithms should approach the true solution as we refine the method.
- **Runtime (Rechenzeit):** measured in the number of elementary operations (additions, multiplications, etc.).
- **Memory consumption (Speicherverbrauch):** some methods may be accurate but infeasible if they require too much storage.
- **Error analysis:** whenever possible, errors should not only be small, but also measurable.

> Note
>
> Runtime in practice correlates with: (i) the *number of operations*, (ii) *memory traffic* (how often data is moved), and (iii) *data locality* (cache-friendly access). We return to these in the memory–layout note below.

3.2 Mathematical Foundation (Linear maps, matrices, complex inner products)

Linear maps. Let V, W be vector spaces over a field $\mathbb{F}$ (typically $\mathbb{R}$ or $\mathbb{C}$). A map $T : V \to W$ is *linear* if, for all $u, v \in V$ and $\lambda \in \mathbb{F}$,

$$T(u+v) = T(u) + T(v), \tag{1}$$

$$T(\lambda u) = \lambda T(u). \tag{2}$$

Linearity makes such maps easy to analyze and compose. Although nature is seldom linear, we frequently *approximate* nonlinear behavior by linear functions, because linearity is tractable both analytically and computationally.

Linearization (first-order approximation). For a scalar function $f : \mathbb{R} \to \mathbb{R}$ differentiable at x_0,

$$f(x) \approx f(x_0) + f'(x_0)\,(x - x_0),$$

which is linear in the variable x around x_0. In multiple dimensions, if $f : \mathbb{R}^n \to \mathbb{R}^m$ is differentiable at x_0,

$$f(x_0 + h) \approx f(x_0) + J_f(x_0)\,h,$$

where $J_f(x_0) \in \mathbb{R}^{m\times n}$ is the Jacobian. The term $h \mapsto J_f(x_0)h$ is a *linear map*.

> Note
>
> Many numerical methods are built around linearization (Newton's method, least squares, PDE discretizations). The quality of the linear model often controls convergence and stability.

Matrices as linear maps. When V, W are finite-dimensional, choosing bases identifies each linear map $T : V \to W$ with a matrix A such that $T(x) = Ax$. Conversely, every matrix $A \in \mathbb{F}^{m\times n}$ defines a linear map $\mathbb{F}^n \to \mathbb{F}^m$. This equivalence is the backbone of numerical linear algebra.

Complex inner products. On $\mathbb{C}^n$, the standard inner product is

$$\langle v, w\rangle = \sum_{i=1}^{n} v_i\,\overline{w_i}.$$

Key properties:

$$\begin{aligned} \text{(Conjugate symmetry)} \quad & \langle v, w\rangle = \overline{\langle w, v\rangle}. \\ \text{(Linearity in the first argument)} \quad & \langle \alpha u + \beta v,\, w\rangle = \alpha\langle u, w\rangle + \beta\langle v, w\rangle, \end{aligned}$$

for all $u, v, w \in \mathbb{C}^n$ and $\alpha, \beta \in \mathbb{C}$. (Consequently, it is *conjugate-linear* in the second argument.)

> Note
>
> Positive definiteness: $\langle v, v\rangle \geq 0$ with equality iff $v = 0$. The induced norm $\|v\|_2 = \sqrt{\langle v, v\rangle}$ is ubiquitous in error and stability analysis.

3.3 Implementation note: Row-major vs. Column-major memory layout

A matrix in memory is ultimately a 1D array. Two common layouts:

- **Row-major (C-order):** consecutive elements from the same *row* are contiguous.
- **Column-major (Fortran-order):** consecutive elements from the same *column* are contiguous.

Note

Why it matters. Contiguous access improves cache locality and enables SIMD/vectorization on the CPU. NumPy uses row-major by default but supports column-major (`order='F'`). Aligning your loops/operations with the underlying layout can yield large speedups.

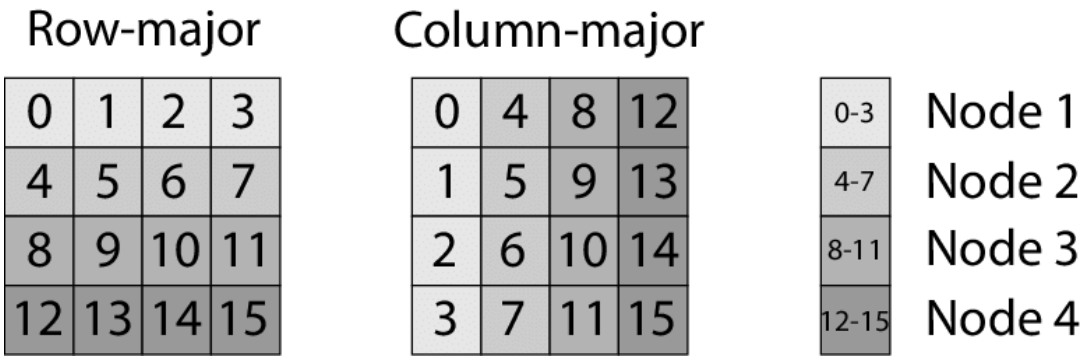

Figure 1: Row-major (C-order) vs. Column-major (Fortran-order).

4 Errors in Numerical Computation

Numerical computation is inherently approximate. Unlike exact arithmetic on paper, computers operate with finite precision, which means not every real number can be represented exactly. This leads to several kinds of *errors* that must be defined and controlled.

4.1 Machine Numbers — Theoretical Detail OPTIONAL

OPTIONAL Optional Reading

This subsection is optional. It provides additional theoretical background that is useful for deeper understanding, but not required for the core weekly goals.

Fix a *base* $B \in \{2, 10, \dots\}$, a number of *digits in the mantissa* $t \in \mathbb{N}$, and an *exponent range* $E \in \{E_{\min}, \dots, E_{\max}\}$ with $E_{\min} < E_{\max}$. A (normalized) t-digit floating-point number has the form

$$x = \pm\,(d_0.d_1 d_2 \dots d_{t-1})_B \; B^E, \qquad d_0 \in \{1, \dots, B-1\}, \quad d_i \in \{0, \dots, B-1\}.$$

Here $d_0 \neq 0$ (except for $x = 0$), and the fractional point indicates that the significand $d_0.d_1 \dots d_{t-1}$ lies in $[1,\, B)$. The corresponding set of *machine numbers* is

$$\mathcal{M} \;=\; \{\,0\,\} \;\cup\; \Big\{ \pm\Big(\sum_{i=0}^{t-1} d_i B^{-i}\Big) B^E \;\Big|\; d_0 \in \{1,\dots,B-1\},\; d_i \in \{0,\dots,B-1\},\; E_{\min} \le E \le E_{\max} \Big\}.$$

Spacing near 1. The gap between 1 and the next larger machine number is

$$\epsilon = B^{\,1-t} \quad \text{(the } \textit{machine epsilon} \text{ in many texts).}$$

For binary IEEE double ($B = 2$, $t = 53$ including the hidden bit), $\epsilon = 2^{-52}$.

Unit roundoff. When rounding to nearest, the relative error of one rounding step is bounded by

$$u = \tfrac{1}{2} B^{\,1-t}.$$

For IEEE double, $u = 2^{-53} \approx 1.11 \times 10^{-16}$. Many authors call u the *unit roundoff* and reserve "machine epsilon" for ϵ; the two differ by a factor ≈ 2.

Extremal numbers. The largest and smallest *normalized* magnitudes in $\mathcal{M}$ are

$$x_{\max} = \big(B - B^{\,1-t}\big)\, B^{E_{\max}}, \qquad x_{\min}^{(\text{normal})} = B^{E_{\min}}.$$

In IEEE double: $x_{\max} \approx 1.7976931348623157 \times 10^{308}$ and $x_{\min}^{(\text{normal})} \approx 2.225074 \times 10^{-308}$. Subnormal (denormal) numbers allow magnitudes down to about $B^{E_{\min}-(t-1)}$; for double that is $2^{-1074} \approx 4.94 \times 10^{-324}$.

Overflow/underflow. If $|x| > x_{\max}$, arithmetic overflows (typically to `Inf`). If $0 < |x| < x_{\min}^{(\text{normal})}$, results are *subnormal* or may *underflow* to 0 depending on the operation and mode.

Note

A floating-point array is stored linearly in memory; spacing of representable numbers is *nonuniform* (coarser as $|x|$ grows). Combined with memory layout (row/column major) this affects performance and error behavior via cache locality and vectorization.

4.2 Absolute and Relative Error

Let $x \in \mathbb{R}$ be the true value and $\hat{x}$ its approximation:

$$e_{\text{abs}} = |x - \hat{x}|, \qquad e_{\text{rel}} = \frac{|x - \hat{x}|}{|x|} \quad (x \neq 0).$$

Absolute error is a raw deviation; relative error is scale-invariant.

Example. If $x = 10^6$ and $\hat{x} = 999{,}999$, then $e_{\text{abs}} = 1$, $e_{\text{rel}} = 10^{-6}$. If $x = 10^{-3}$ and $\hat{x} = 2 \cdot 10^{-3}$, then $e_{\text{abs}} = 10^{-3}$, $e_{\text{rel}} = 1$.

Note

At or near $x = 0$, use absolute errors or a scaled denominator such as $\max\{1, |x|\}$ depending on context.

4.3 Rounding Error and the Roundoff Model

Correct rounding. The rounding map $\text{rd} : \mathbb{R} \to \mathcal{M}$ sends a real x to a nearest machine number:

$$\text{rd}(x) \in \arg\min_{y \in \mathcal{M}} |x - y|,$$

with a specified tie-breaking rule (IEEE: *round to nearest, ties to even*).

Axiom of roundoff analysis. For elementary operations $\circ \in \{+, -, \times, /\}$ (and many "hard-wired" functions),

$$\text{fl}(x \circ y) = (x \circ y)\,(1 + \delta), \qquad |\delta| \leq u,$$

and for unary f,

$$\text{fl}(f(x)) = f(x)\,(1 + \delta), \qquad |\delta| \leq u,$$

where u is the unit roundoff. This idealized model is the standard tool for error estimates.

Note

Due to rounding, addition/multiplication are not associative: $(x+y)+z \neq x+(y+z)$. Avoid equality tests like `if (x == 0)`; use `abs(x) < tol` with a problem-aware tolerance, typically scaled by u.

4.4 Error Accumulation

Consider

$$S = \sum_{i=1}^{n} x_i, \qquad \hat{S} = \mathrm{fl}\left(\sum_{i=1}^{n} x_i\right).$$

If each addition incurs a relative error $\leq u$, a crude worst-case bound is

$$|\hat{S} - S| \lesssim n\,u \max_i |x_i|.$$

Large n and unfavorable summation order can make roundoff visible even when u is tiny.

4.5 Cancellation

Definition. *Cancellation* is the loss of leading significant digits that occurs when combining (nearly) equal quantities—typically in a subtraction $x - y$ or an addition of opposite signs. The true result is small compared to the operands, so tiny perturbations dominate it.

Note

Many students confuse **rounding error** with **cancellation**. *Rounding error* is the *local* perturbation introduced by one floating-point step, modeled by $\mathrm{fl}(x \circ y) = (x \circ y)(1+\delta)$ with $|\delta| \leq u$ (unit roundoff). *Cancellation* is an *amplification mechanism*: even tiny rounding/measurement errors can produce a large *relative* error in the result of a nearly-exact subtraction.

A simple estimate makes the distinction precise. With

$$\hat{x} = x(1+\delta_1), \quad \hat{y} = y(1+\delta_2), \quad |\delta_i| \leq u, \qquad \hat{z} = \mathrm{fl}(\hat{x} - \hat{y}),$$

one obtains (to first order)

$$\frac{|\hat{z} - (x-y)|}{|x-y|} \approx \underbrace{\frac{|x| + |y|}{|x-y|}}_{\kappa_{\mathrm{sub}}} u,$$

where κ_{sub} is the *condition number of subtraction*. When $x \approx y$, κ_{sub} is large and cancellation becomes catastrophic.

Cancellation: examples and remedies

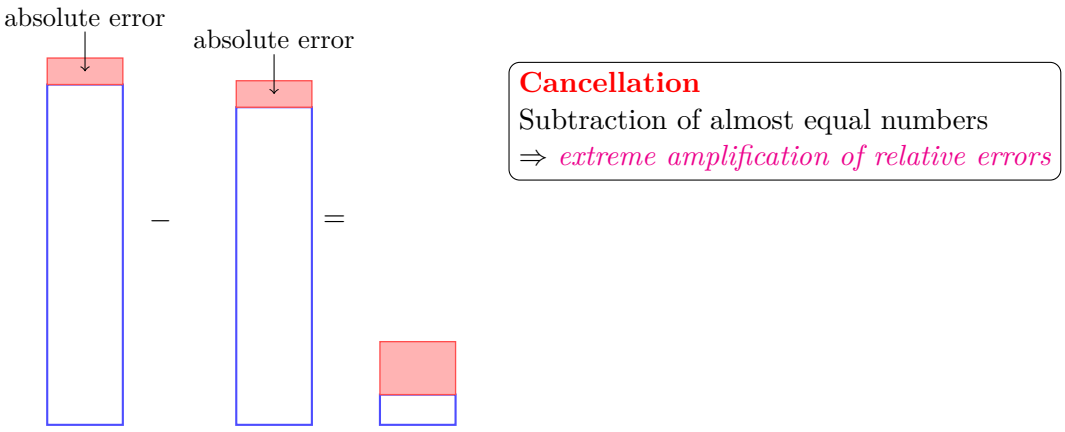

Figure 2: Visualization of cancellation: two large, almost equal numbers each with small absolute errors (red). The subtraction yields a small result, but the error dominates it, leading to a huge relative error.

1) Numerical differentiation. **Goal.** We want the slope $f'(x)$. A simple idea is the slope of the secant:

$$D_h f(x) = \frac{f(x+h) - f(x)}{h}.$$

If h is small, the secant looks like the tangent, so $D_h f(x)$ *should* be close to $f'(x)$.

Two independent error sources.

1) *Truncation (model) error:* even with exact arithmetic, a secant is not the tangent. A short Taylor expansion says

$$f(x+h) = f(x) + hf'(x) + \tfrac{h^2}{2} f''(x) + \cdots \quad \Rightarrow \quad D_h f(x) = f'(x) + \tfrac{h}{2} f''(x) + O(h^2).$$

So the *model* error is proportional to h (smaller h helps).

2) *Rounding (cancellation) error:* in floating-point, each value is stored with tiny relative error $\sim u$ (unit roundoff). When h is very small, $f(x+h)$ and $f(x)$ are *almost equal*. Their subtraction

$$f(x+h) - f(x)$$

loses many leading digits (cancellation). After dividing by a tiny h, this rounding noise is *amplified* by $1/h$.

Note

Which formula to use?

- *Central difference* $\dfrac{f(x+h)-f(x-h)}{2h}$ has smaller model error ($O(h^2)$), so the optimal h is larger (safer).

  Taylor expansions:

  $$f(x+h) = f(x) + hf'(x) + \tfrac{h^2}{2}f''(x) + \tfrac{h^3}{6}f^{(3)}(x) + \dots$$

  $$f(x-h) = f(x) - hf'(x) + \tfrac{h^2}{2}f''(x) - \tfrac{h^3}{6}f^{(3)}(x) + \dots$$

  - **Forward difference:**

  $$\frac{f(x+h)-f(x)}{h} = f'(x) + \tfrac{h}{2}f''(x) + O(h^2),$$

  so the truncation error is $O(h)$.

  - **Central difference:**

  $$\frac{f(x+h)-f(x-h)}{2h} = f'(x) + \tfrac{h^2}{6}f^{(3)}(x) + O(h^4),$$

  so the truncation error is $O(h^2)$.
- *Complex-step* (see item 7) avoids subtraction altogether and has no u/h blow-up, so you can take h extremely small and reach near machine precision if f accepts complex inputs.

2) Quadratic roots . For $\varphi(x) = x^2 + \alpha x + \beta$ ($\Delta = \alpha^2 - 4\beta$), the naive

$$x_{1,2} = \frac{-\alpha \pm \sqrt{\Delta}}{2}$$

suffers cancellation when $-\alpha$ and $+\sqrt{\Delta}$ are close.

Stable variant (Viète):

$$q \;=\; -\frac{\alpha + \operatorname{sign}(\alpha)\sqrt{\Delta}}{2}, \qquad x_1 = q, \qquad x_2 = \frac{\beta}{q}.$$

Numeric example. Take $\alpha = 10^8$, $\beta = 1$. Then $\Delta = 10^{16} - 4$ and

$$\sqrt{\Delta} = 10^8\sqrt{1 - 4\cdot 10^{-16}} \approx 10^8\left(1 - 2\cdot 10^{-16}\right) = 10^8 - 2\cdot 10^{-8}.$$

The *small* root is

$$x_{\text{small}} = \frac{-\alpha + \sqrt{\Delta}}{2} \approx \frac{-10^8 + (10^8 - 2\cdot 10^{-8})}{2} = -10^{-8}.$$

However, near 10^8 the spacing between representable doubles is about ULP $\approx 10^8 \cdot u \approx 10^{-8}$.

Thus the subtraction $(-10^8) + (10^8 - 2 \cdot 10^{-8})$ loses almost all digits: the *relative* error of x_{small} can be $O(1)$. Using the stable variant, compute the large root $q = \frac{-\alpha - \sqrt{\Delta}}{2} \approx -10^8$ accurately, then the small one as $x_2 = \beta/q \approx -10^{-8}$ with full precision.

3) $1 - \cos x$ **for small** x**.** Direct evaluation loses many significant digits. *Remedy:*

$$1 - \cos x \;=\; 2\sin^2\!\left(\frac{x}{2}\right),$$

which avoids subtracting two nearly equal numbers.

4) Difference of square roots. For $y = \sqrt{1+x} - \sqrt{1-x}$ (cancellation when x is small),

$$y = \frac{(1+x) - (1-x)}{\sqrt{1+x} + \sqrt{1-x}} = \frac{2x}{\sqrt{1+x} + \sqrt{1-x}},$$

a stable reformulation.

5) $I(a) = \dfrac{e^a - 1}{a}$ **near** $a = 0$**.** The numerator cancels for small a.

$$I(a) = \int_0^1 e^{at}\, dt \quad \text{and} \quad I(a) = \sum_{k=0}^{\infty} \frac{a^k}{(k+1)!} = 1 + \frac{a}{2} + \frac{a^2}{6} + \cdots .$$

In practice use the stable primitive $\operatorname{expm1}(a) = e^a - 1$ and compute

$$I(a) = \frac{\operatorname{expm1}(a)}{a},$$

or the series truncated at $k = m$ with remainder bounded by $\lesssim \dfrac{e^{|a|}\,|a|^{m+1}}{(m+2)!}$.

> Note
>
> Most math libraries provide `expm1`. It evaluates $e^a - 1$ without cancellation.

6) Projection onto a direction (near–parallel case). Let $a, b \in \mathbb{R}^n$, $b \neq 0$. The projection of a onto $\operatorname{span}\{b\}$ is

$$\operatorname{proj}_b(a) \;=\; \frac{a^\top b}{b^\top b}\, b,$$

and the *projection residual* (the component of a orthogonal to b) is

$$p \;=\; a \;-\; \frac{a^\top b}{b^\top b}\, b.$$

If a is nearly parallel to b (small angle θ), then p is *very small*:

$$\|p\| \;=\; \|a\| \sin\theta \;\approx\; \|a\|\,\theta \quad (\theta \ll 1).$$

Computing p naively subtracts two large, close vectors — classic *cancellation*. A few ulps of rounding in the dot product $a^\top b$ or in the scaled vector $(\frac{a^\top b}{b^\top b})b$ can dominate the tiny result p.

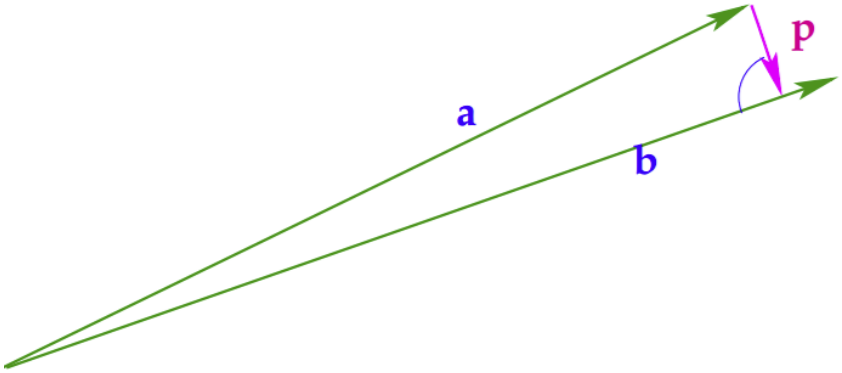

Figure 3: Projection of a onto b in the near-parallel case. The residual $p = a - \text{proj}_b(a)$ is orthogonal to b and small in norm for $\theta \ll 1$.

Note

Why this is ill-conditioned. The relative error of p scales like $1/\sin\theta$. When θ is tiny, even tiny rounding can cause large relative error in p.

7) Complex-step differentiation. Assume f extends to a complex-analytic function near $x \in \mathbb{R}$. For a tiny real $h > 0$,

$$f(x + ih) = f(x) + ihf'(x) - \frac{h^2}{2}f''(x) - i\frac{h^3}{6}f^{(3)}(x) + O(h^4).$$

Taking the imaginary part and dividing by h gives

$$\boxed{f'(x) \approx \frac{\Im f(x + ih)}{h}} \quad \text{with truncation error } O(h^2).$$

Why it avoids cancellation. No subtraction of close real values occurs; we read the derivative directly from the *imaginary part*. Thus the rounding term does *not* scale like u/h.

How to use it (practical steps).

1) Pick a tiny h (e.g. $h = 10^{-20}$ in double precision).

2) Compute $f(x + ih)$ using complex-aware math functions.

3) Return $\Im f(x + ih)/h$ as the derivative.

Note

Advantages. No subtractive cancellation; you can take h extremely small and get near machine precision (limited by the library). **Works when** f is built from smooth, complex-aware ops (`+ - * / exp sin cos log ...`). **Avoid** non-analytic ops at the evaluation point (e.g. `abs`, `max`, branching on sign), which break the trick.

Note

Rule of thumb. Whenever you see a *difference of close quantities*, look for an identity (trigonometric/algebraic), a Viète/rationalization trick, or an integral/series representation that avoids the dangerous subtraction.

5 Computational Effort (Cost) vs. Run Time

5.1 What do we measure?

Traditional view. We count *elementary operations*: additions, subtractions, multiplications, divisions, square-roots, etc. This gives a machine-independent notion of cost (e.g. "$2n^3$ flops" for dense $n \times n$ matrix multiply).

Modern reality. *Computational effort* (op count) is not the same as *run time*. On contemporary hardware, run time is often dominated by *memory traffic*: fetching/storing data through caches and main memory. (See *DDCA*, *SPCA*, *PProg* courses for details.) Thus:

$$\text{run time} \approx \max\{\text{compute bound}, \text{ memory bound}\}.$$

> Note
>
> Cache effects can strongly distort timing curves for small and medium sizes n; only for large n does the asymptotic behavior stabilize. We therefore analyze costs asymptotically, $\text{cost}(n) = \Theta(n^\beta)$ as $n \to \infty$, but we also care about *arithmetic intensity* (ops per byte moved) and *data locality*.

5.2 Core dense operations (cost model)

Below, "cost" is the leading-term op count (flops) and $\Theta(\cdot)$ class. For rectangular sizes, we include general formulas. Entries marked † are *optimal for general inputs*: you must at least read all inputs and write all outputs, so any algorithm is Ω of that size.

Operation	Definition	Cost (flops / Θ)
Dot product †	Given $x, y \in \mathbb{R}^n$, $x^\top y = \sum_{i=1}^{n} x_i y_i$.	n multiplies + $(n-1)$ adds $\approx 2n$ flops; $\Theta(n)$
Tensor (outer) product †	Given $u \in \mathbb{R}^m$, $v \in \mathbb{R}^n$, $u \otimes v = u\,v^\top \in \mathbb{R}^{m \times n}$ (rank-1 matrix).	mn multiplies (and mn writes); $\Theta(mn)$
Matrix–vector multiply †	$A \in \mathbb{R}^{m \times n}$, $x \in \mathbb{R}^n$, $y = Ax \in \mathbb{R}^m$. Each row of A dotted with x.	Per row: (n mults $+(n-1)$ adds); total $\approx 2mn$ flops; $\Theta(mn)$
Matrix–matrix multiply	$A \in \mathbb{R}^{m \times n}$, $B \in \mathbb{R}^{n \times p}$, $C = AB \in \mathbb{R}^{m \times p}$.	Naïve (three loops): $\approx 2mnp$ flops; $\Theta(mnp)$. For $n \times n$: $\approx 2n^3$ flops; $\Theta(n^3)$

† *Optimal for general inputs:* you must at least read all entries of the inputs and write all entries of the outputs (I/O lower bound), so any algorithm is $\Omega(n)$, $\Omega(mn)$, $\Omega(mn)$ respectively.

Note

Memory layout matters. Under the hood, every matrix is a 1D array. Row-major vs. column-major, contiguity (strides), and blocking determine cache reuse and thus run time. High-performance libraries use *blocked* algorithms to increase arithmetic intensity, keeping data in fast caches.

5.3 Case 1 — Associativity trick (rank-1 × vector)

Let $a, b, x \in \mathbb{R}^n$ and consider $y = (a\,b^\top)\,x$.

Naïve (bad) parenthesization. Form the outer product $C = a\,b^\top \in \mathbb{R}^{n\times n}$, then $y = Cx$. This costs $\Theta(n^2)$ flops and writes n^2 entries.

Smart (good) parenthesization. Use associativity of matrix multiplication:

$$(a\,b^\top)\,x \;=\; a\,(b^\top x).$$

Compute the scalar $s = b^\top x$ in $\approx 2n$ flops, then $y = a\,s$ in n flops: overall $\Theta(n)$.

Note

Optimality. Any algorithm must at least read all entries of a, b, x, so the problem has an I/O lower bound $\Omega(n)$. The associative form achieves $\Theta(n)$ and avoids creating an $n \times n$ matrix.

5.4 Case 2 — Hidden Summation (matrix view): $y = \mathrm{triu}(AB^\top)\,x$

Let $A, B \in \mathbb{R}^{n\times p}$ with $p \ll n$, $x \in \mathbb{R}^n$, and $\mathrm{triu}(\cdot)$ keep only the upper triangular part:

$$\big[\mathrm{triu}(M)\big]_{ij} = \begin{cases} M_{ij}, & i \le j, \\ 0, & i > j. \end{cases}$$

Warm-up: $p = 1$ (rank-1 case). Write $A = a \in \mathbb{R}^n$ and $B = b \in \mathbb{R}^n$. Then $AB^\top = a\,b^\top$ and

$$y \;=\; \mathrm{triu}(a\,b^\top)\,x.$$

Define the *upper-replicating* matrix of b:

$$U(b) = \begin{bmatrix} b_1 & b_2 & b_3 & \cdots & b_n \\ 0 & b_2 & b_3 & \cdots & b_n \\ 0 & 0 & b_3 & \cdots & b_n \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & b_n \end{bmatrix}.$$

Observe

$$\operatorname{triu}(a\,b^\top) = \operatorname{diag}(a)\,U(b), \qquad \text{so} \quad y = \operatorname{diag}(a)\,U(b)\,x.$$

Multiplying out the i-th component:

$$y_i \;=\; a_i \sum_{j=i}^{n} b_j\,x_j \quad\Longleftrightarrow\quad \boxed{y \;=\; a \;\odot\; S\;,\;\; S_i = \sum_{j=i}^{n} (b_j x_j)\;,}$$

where $\odot$ denotes elementwise (Hadamard) product.

Efficient computation (no $n \times n$ matrices):

$$\begin{aligned}
&(1)\ w \leftarrow b \odot x && \text{cost } \Theta(n)\\
&(2)\ S \leftarrow \text{reverse-cumsum of } w:\ S_i = \sum_{j=i}^{n} w_j && \Theta(n)\\
&(3)\ y \leftarrow a \odot S && \Theta(n)
\end{aligned}$$

Total cost $\Theta(n)$ and $O(1)$ extra memory (if you do the reverse cumsum in place).

Small $n = 4$ picture.

$$\operatorname{triu}(a\,b^\top) = \begin{bmatrix} a_1 & 0 & 0 & 0\\ 0 & a_2 & 0 & 0\\ 0 & 0 & a_3 & 0\\ 0 & 0 & 0 & a_4 \end{bmatrix} \begin{bmatrix} b_1 & b_2 & b_3 & b_4\\ 0 & b_2 & b_3 & b_4\\ 0 & 0 & b_3 & b_4\\ 0 & 0 & 0 & b_4 \end{bmatrix}, \quad U(b)\,x = \begin{bmatrix} b_1x_1 + b_2x_2 + b_3x_3 + b_4x_4\\ b_2x_2 + b_3x_3 + b_4x_4\\ b_3x_3 + b_4x_4\\ b_4x_4 \end{bmatrix}.$$

Note

Optimality. Any algorithm must at least read a, b, x and write y ($\Omega(n)$). The scheme above achieves the lower bound and avoids materializing $U(b)$ or $a\,b^\top$.

General case: $p > 1$ (low rank). Write $A = [a^{(1)} \;\cdots\; a^{(p)}]$, $B = [b^{(1)} \;\cdots\; b^{(p)}]$ with columns $a^{(k)}, b^{(k)} \in \mathbb{R}^n$. Then

$$AB^\top = \sum_{k=1}^{p} a^{(k)}\,b^{(k)\top} \quad\Longrightarrow\quad \operatorname{triu}(AB^\top) = \sum_{k=1}^{p} \operatorname{triu}\big(a^{(k)}\,b^{(k)\top}\big).$$

Applying the rank-1 result to each term,

$$\boxed{y \;=\; \sum_{k=1}^{p} a^{(k)} \odot S^{(k)}, \qquad S_i^{(k)} = \sum_{j=i}^{n} b_j^{(k)}\,x_j.}$$

Efficient algorithm ($\Theta(np)$, $O(n)$ memory):

$$
\begin{aligned}
&y \leftarrow 0 \\
&\text{for } k = 1, \dots, p: \\
&\qquad w \leftarrow b^{(k)} \odot x && \Theta(n) \\
&\qquad S \leftarrow \text{reverse-cumsum}(w) && \Theta(n) \\
&\qquad y \leftarrow y + a^{(k)} \odot S && \Theta(n)
\end{aligned}
$$

Total cost $\Theta(np)$; you only keep one temporary vector w and S.

Note

Why $\Theta(np)$ is optimal. For dense inputs, you must at least read all entries of A and B (each np numbers) and the vector x (n), and write y (n). Thus any algorithm is $\Omega(np)$; the scheme above matches this bound up to constants.

5.5 Case 3 — Reuse of intermediate results (Kronecker × vector)

Setup. Given $A \in \mathbb{R}^{m\times n}$ and $B \in \mathbb{R}^{k\times \ell}$, their Kronecker product is the block matrix

$$
A \otimes B = \begin{bmatrix} A_{11}B & A_{12}B & \cdots & A_{1n}B \\ A_{21}B & A_{22}B & \cdots & A_{2n}B \\ \vdots & \vdots & \ddots & \vdots \\ A_{m1}B & A_{m2}B & \cdots & A_{mn}B \end{bmatrix}, \quad \in \mathbb{R}^{(mk)\times(n\ell)}.
$$

We want to compute

$$
y = (A \otimes B)\, x, \qquad x \in \mathbb{R}^{n\ell},\ y \in \mathbb{R}^{mk}.
$$

Block view. Partition x into n blocks $x^{(j)} \in \mathbb{R}^{\ell}$:

$$
x = \begin{bmatrix} x^{(1)} \\ x^{(2)} \\ \vdots \\ x^{(n)} \end{bmatrix}.
$$

Then multiplication with $(A \otimes B)$ gives

$$
y = \begin{bmatrix} \sum_{j=1}^{n} A_{1j}\,(Bx^{(j)}) \\ \sum_{j=1}^{n} A_{2j}\,(Bx^{(j)}) \\ \vdots \\ \sum_{j=1}^{n} A_{mj}\,(Bx^{(j)}) \end{bmatrix}.
$$

Visualization (small $m = n = 2$).

$$A \otimes B = \begin{bmatrix} A_{11}B & A_{12}B \\ A_{21}B & A_{22}B \end{bmatrix}, \qquad x = \begin{bmatrix} x^{(1)} \\ x^{(2)} \end{bmatrix}.$$

So

$$y = \begin{bmatrix} A_{11}(Bx^{(1)}) + A_{12}(Bx^{(2)}) \\ A_{21}(Bx^{(1)}) + A_{22}(Bx^{(2)}) \end{bmatrix}.$$

Reuse of intermediate results. The expensive pieces are the block products $Bx^{(j)} \in \mathbb{R}^k$.

$$w^{(j)} := Bx^{(j)}.$$

They can be computed once for each j and reused across all rows of A:

$$y^{(i)} = \sum_{j=1}^{n} A_{ij}\, w^{(j)}, \qquad i = 1, \ldots, m,$$

where $y^{(i)} \in \mathbb{R}^k$ is the i-th block of y.

Cost. - Compute $w^{(j)} = Bx^{(j)}$ once: $\Theta(nk\ell)$ - Recombine with A: $\Theta(mnk)$ Total: $\Theta(nk(\ell+m))$ versus $\Theta(mnk\ell)$ if forming $A \otimes B$ explicitly.

Note

Optimality. Every entry of A, B, x must be read and every entry of y written, so the scheme is essentially optimal and avoids building the giant $(mk) \times (n\ell)$ matrix.